

Project Management System with AI

Chat and Calling Integration Proposal

Version 1.0 | 30 September 2026 | Pending approval

Approval requested

We request approval to integrate a communication module into Project Management System with AI. The module will provide project group chat, direct messages, file sharing, and one to one voice and video calls within the existing system. This document explains the current prototype, the technologies used, and the work required for integration.

This is a proposal for review. No integration into the Project Management System is being performed as part of this document. Development will begin after approval and confirmation of the target system architecture.

Purpose and expected benefit

Project members should be able to discuss tasks, clarify requirements, share supporting files, and call a colleague without leaving the project workspace. Conversations should be connected to the relevant project and accessible only to permitted members.

Scope for the first approved release

- Project group chat and private conversations between permitted users.
- Member list, online status, unread counts, timestamps and responsive desktop, tablet and mobile screens.
- File attachments with an agreed size limit and controlled download access.
- One to one voice and video calls with accept, decline, mute, camera toggle and end call controls.
- Integration with PMS login, project membership, message history and approved storage.

Scope excluded from the first release

Group video conferencing, screen sharing, call recording, transcription, native mobile applications and AI chat summaries are not included in the base proposal. These can be reviewed separately. The communication prototype currently has no AI integration.

Basis of the proposal

The existing WebApplication2 chat project is the reference prototype. The target PMS codebase, database, login mechanism, deployment environment, user volume and AI provider have not been supplied or reviewed. Integration choices below are recommendations, not confirmed properties of that system.

1 Technology used in the prototype

SignalR carries chat messages and call setup events. WebRTC carries the live audio and video. These are separate responsibilities: audio and video streams are not sent through the SignalR chat hub. [1, 2]

Component	Role in this module
ASP.NET Core MVC and C#	Hosts the web application, serves the Razor UI, validates requests and exposes the chat hub. The prototype targets .NET 9.
ASP.NET Core SignalR	Delivers group and private messages, presence updates, call invitations and WebRTC signaling. The local browser client is version 9.0.6.
WebSockets and JSON	SignalR prefers WebSockets and can fall back to other supported transports. JSON represents the hub messages. [1]
WebRTC and RTCPeerConnection	Establishes the browser media connection for one to one voice and video calls. [2]
getUserMedia and MediaStream	Requests microphone or camera access and provides local media tracks. User permission and a secure browser context are required. [3]
ICE and STUN	ICE discovers connection routes. The prototype configures Google STUN at stun:stun.l.google.com:19302. [2]
TURN service	Relays media when direct connectivity is unavailable. Configuration support exists, but no TURN server or credentials are supplied. [2]
Razor HTML CSS and JavaScript	Builds the responsive chat UI, conversation selection, message rendering, attachment handling and call controls.
FileReader Blob and object URLs	Reads a selected file, encodes it as Base64 for SignalR delivery and creates a downloadable attachment in the recipient browser.
Browser memory and localStorage	Messages and drafts remain in the tab memory. localStorage remembers only the display name and theme preference.

Runtime and dependency decision

The server SignalR framework is included with ASP.NET Core; it does not require the old standalone SignalR server package. The browser library is bundled locally with its MIT license. For PMS integration, select a maintained runtime and compatible packages after reviewing the target project; the prototype version is a reference, not a required deployment version.

2 How chat files and calls work

Group and private chat

A visitor joins with a chosen display name. The hub associates that name with the current connection ID and publishes the connected users. A group message goes to all joined users in the prototype. A private message goes only to the sender and selected recipient connection. The server determines the sender from the connection rather than trusting a sender name in the message.

For PMS integration, the shared room must become a project specific conversation. The server must verify project membership before joining the room or delivering a message. Logged in user IDs must replace temporary connection IDs as the lasting identity.

File sharing

The browser reads the attachment and sends its Base64 data with the message. The hub checks the filename, decoded size and data format before routing it. The recipient gets a download link created from a Blob. Files are not saved on the server in the prototype.

For PMS, use an authenticated upload endpoint and private file storage. Save attachment metadata with the message and send a file reference through SignalR. Recheck access when downloading. This avoids carrying large files through the live chat connection and supports history after refresh.

Voice and video call sequence

1. The caller selects a person and starts a voice or video call. The browser asks for the required media permission.
2. SignalR sends a call invitation to the recipient. The recipient may accept or decline; busy users cannot receive a second simultaneous call.
3. After acceptance, the browsers exchange an SDP offer, SDP answer and ICE candidates through the hub. SDP describes media settings; ICE candidates describe possible network routes.
4. WebRTC establishes the media path directly between browsers where possible, or through a configured TURN relay when required.
5. Either participant can mute the microphone, toggle the camera on a video call, or end the call. Ending or disconnecting closes the peer connection and stops local media tracks.

Communication paths

Chat and call setup: Browser A → SignalR server → Browser B

Live audio and video: Browser A ↔ WebRTC connection ↔ Browser B

If relay is required: Browser A ↔ TURN server ↔ Browser B

SignalR does not provide video conferencing by itself. WebRTC supplies the media connection, while the application manages invitation, authorization and call lifecycle. [1, 2]

3 Required changes for PMS integration

Identity and project permissions

Use the existing PMS authentication mechanism and verified user profile. Map each user to their active connections so refreshes and multiple tabs do not create a new person. Verify conversation membership on every send, history request, file download and call operation. A project role alone should not grant access to a private conversation without an explicit policy.

Proposed data model

Reuse existing user and project records. The following entities are a suggested design; final naming and database technology must follow the PMS architecture.

Entity	Principal information
Conversation	ID, project ID where applicable, group or direct type, created time.
ConversationMember	Conversation ID, user ID, membership state and last read message or time.
Message	ID, conversation ID, authenticated sender ID, text, sent time and client request ID for deduplication.
Attachment	ID, message ID, original name, storage key, size, type and scan status.
CallSession	ID, conversation or participant references, voice or video type, timestamps and outcome. Metadata only; no recording.

Message history and reliable reconnect

Persist messages before acknowledging success. Use stable message IDs, paginated history and a defined retry policy to prevent duplicate sends. On reconnect, restore permitted memberships and fetch missed messages. Define separately what sent, delivered and read mean; unread counters in the prototype are not read receipts.

Security and file access

Require authenticated hub access and HTTPS. Enforce authorization on the server, render text safely, validate file type and size, scan uploads, and keep stored attachments private. Add quotas and rate limits for messages, uploads and calls. Do not log message bodies, file content, session tokens or TURN secrets by default.

The prototype routes private messages to two connections, but has no verified login, persistent authorization or text and file end to end encryption. Transport protection is not the same as end to end encryption. Approval of additional encryption, retention and deletion policies is required before making related product claims.

Proposed AI boundary

The chat module can work independently of AI. Optional future features include conversation summaries and suggested tasks. They require separate approval of the AI provider, eligible conversations, data handling and user permissions. Any suggested task should require user review before creation. No chat, file or call content will be sent to an AI service under this base proposal.

4 Hosting limits and delivery plan

Deployment requirements

Provide an HTTPS endpoint trusted by each device, a reverse proxy configured for SignalR, compatible browsers, and microphone or camera hardware for calls. HTTP on a phone using a LAN IP is not sufficient for media capture. Localhost can be used for local development. [3]

Provision and test a TURN service for reliable calls across restrictive networks. Use scoped, short lived credentials issued to authorized users. Hosting, storage, backup, scanning and TURN bandwidth may incur operating costs; no provider, budget or capacity commitment is made in this proposal.

For multiple application instances, design shared presence and call state plus SignalR scale out. A backplane or managed SignalR service alone does not replace the prototype's in memory dictionaries. Confirm expected concurrent users and simultaneous calls before capacity planning.

Prototype limits to consider

Item	Current behavior
Names and presence	Guest chosen names of 2–30 characters; duplicate online names rejected. All connected clients can receive the presence list.
Text and sending	Up to 4,000 characters; minimum 500 ms between send attempts per connection.
Attachments	Up to 2 MiB decoded per file; 3 MiB hub receive limit accommodates Base64 overhead.
History	Tab memory only, capped at 500 messages or 24 MiB of attachments. Refresh clears it; no offline delivery.
Calls	One to one only; call setup timeout and disconnect cleanup. No group conference, recording or transcription.
Server state	Single process dictionaries; active presence and call state are lost on restart.

Delivery after approval

Phase 1 Review the PMS codebase, authentication, permissions, database and hosting; agree the schema and UI entry point.

Phase 2 Integrate project rooms, direct messages, authorized presence and persistent history.

Phase 3 Add controlled file uploads and downloads; connect voice and video signaling to authenticated users.

Phase 4 Configure HTTPS and TURN, run functional and security tests, complete user acceptance testing and prepare deployment instructions.

Effort and schedule will be estimated after the target system review. Approval of this proposal does not imply that the prototype can be copied into production without adaptation.

5 Validation and approval

Existing evidence and remaining checks

During the earlier prototype work, the build completed with zero warnings and errors. Automated checks covered group and private routing, attachment validation, the 2 MiB boundary, duplicate names, call signaling authorization, busy handling and disconnect cleanup. Desktop, phone and tablet layouts were inspected. These are prototype results, not PMS acceptance results.

Real microphone and camera calls between two devices across different networks have not yet been verified. Successful signaling tests do not prove media quality or TURN connectivity. PMS integration must pass the following acceptance checks before release.

- Unauthorized users cannot read, send, download or call across project or private conversation boundaries.
- Messages and permitted attachments remain available after refresh; reconnect restores missed history without duplicates.
- Voice and video work in both directions on the agreed desktop and mobile browsers, including a TURN relay test.
- Permission denial, device absence, decline, busy state, timeout, network interruption and hangup behave clearly; media capture stops at call end.
- Attachment rules, retention, access revocation and deletion are verified; the UI remains usable on phone, tablet and desktop.
- Load tests meet agreed concurrent user and call targets, with logs and monitoring sufficient to investigate failures.

Decisions requested from the reviewer

Please confirm the first release scope, eligible project members and direct message policy, attachment limit, history retention period, target hosting environment, expected usage, and approval to provision TURN and storage. AI features and group conferencing remain separate decisions.

Decision Approve proposed scope / Approve with changes / Defer

Reviewer name _____

Comments _____

Signature and date _____

Source basis and technical references

Prototype reviewed on 30 September 2026: WebApplication2.csproj, Program.cs, Hubs/ChatHub.cs, Controllers/HomeController.cs, Views/Home/Index.cshtml, wwwroot/js/site.js, wwwroot/css/site.css, appsettings.json, README.md and tests/chat-smoke.mjs. The proposed PMS design is not an implementation claim.

[1] Microsoft Learn ASP.NET Core SignalR overview [Open reference](#)

[2] MDN Signaling and video calling [Open reference](#)

[3] MDN MediaDevices getUserMedia [Open reference](#)